

Vibecoding With ALCF Inference

Programming with open-weight models on ALCF hardware

Ethan Wong

September 29, 2026



Argonne National Laboratory is a
U.S. Department of Energy laboratory
managed by UChicago Argonne, LLC.



Argonne Leadership
Computing Facility



U.S. DEPARTMENT
of ENERGY

Argonne National Laboratory is a
U.S. Department of Energy laboratory
managed by UChicago Argonne, LLC.

Introduction



U.S. DEPARTMENT
of ENERGY

Argonne National Laboratory is a
U.S. Department of Energy laboratory
managed by UChicago Argonne, LLC.



Argonne Leadership
Computing Facility

What is Vibecoding?

- Term coined by Andrej Karpathy (Feb 2025):

What is Vibecoding?

- Term coined by Andrej Karpathy (Feb 2025):
“There’s a new kind of coding I call ‘vibe coding’, where you fully give in to the vibes, embrace exponentials and forget that the code even exists.”

What is Vibecoding?

- Term coined by Andrej Karpathy (Feb 2025):
“There’s a new kind of coding I call ‘vibe coding’, where you fully give in to the vibes, embrace exponentials and forget that the code even exists.”
- **In practice:** you describe what you want in plain language and a coding agent writes, runs, and debugs the code. You iterate by reviewing the results and re-prompting instead hand-writing code.

What is Vibecoding?

- Term coined by Andrej Karpathy (Feb 2025):
“There’s a new kind of coding I call ‘vibe coding’, where you fully give in to the vibes, embrace exponentials and forget that the code even exists.”
- **In practice:** you describe what you want in plain language and a coding agent writes, runs, and debugs the code. You iterate by reviewing the results and re-prompting instead hand-writing code.
- Note that this doesn’t absolve critical-thinking! You still need to do the hard work of discerning what to make, how robust it’s built, and judging whether it is correct.

What is Vibecoding?

- Term coined by Andrej Karpathy (Feb 2025):
“There’s a new kind of coding I call ‘vibe coding’, where you fully give in to the vibes, embrace exponentials and forget that the code even exists.”
- **In practice:** you describe what you want in plain language and a coding agent writes, runs, and debugs the code. You iterate by reviewing the results and re-prompting instead hand-writing code.
- Note that this doesn’t absolve critical-thinking! You still need to do the hard work of discerning what to make, how robust it’s built, and judging whether it is correct.
- For domain scientists: the barrier shifts from syntax and software-development expertise to specification and verification, skills you already have.

From Chatbots to Coding Agents

- **2021–22:** IDE autocomplete brings Copilot suggestions into your editor, still line-by-line

From Chatbots to Coding Agents

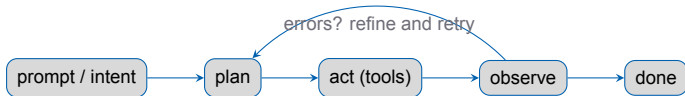
- **2021–22:** IDE autocomplete brings Copilot suggestions into your editor, still line-by-line
- **2022:** Chat assistants. You copy/paste snippets from the chat window, run everything, and fix every error

From Chatbots to Coding Agents

- **2021–22:** IDE autocomplete brings Copilot suggestions into your editor, still line-by-line
- **2022:** Chat assistants. You copy/paste snippets from the chat window, run everything, and fix every error
- **2024–25:** Terminal agents that read/write files, run shell commands, search the repo, observe errors, and self-correct

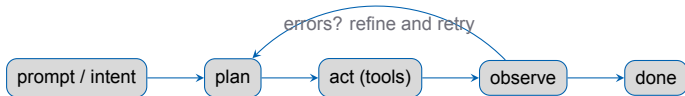
From Chatbots to Coding Agents

- **2021–22:** IDE autocomplete brings Copilot suggestions into your editor, still line-by-line
- **2022:** Chat assistants. You copy/paste snippets from the chat window, run everything, and fix every error
- **2024–25:** Terminal agents that read/write files, run shell commands, search the repo, observe errors, and self-correct



From Chatbots to Coding Agents

- **2021–22:** IDE autocomplete brings Copilot suggestions into your editor, still line-by-line
- **2022:** Chat assistants. You copy/paste snippets from the chat window, run everything, and fix every error
- **2024–25:** Terminal agents that read/write files, run shell commands, search the repo, observe errors, and self-correct



The agent is now the executor. You now focus on the overall architecture and review.

Anatomy of a Coding Agent

- An **agent harness** is this read-eval-print loop as one coherent piece of software¹ with:

¹Exactly what you built in the previous session!

Anatomy of a Coding Agent

- An **agent harness** is this read-eval-print loop as one coherent piece of software¹ with:
 - **Tools:** read/edit/write files, run shell commands, search the codebase, (sometimes) the web

¹Exactly what you built in the previous session!

Anatomy of a Coding Agent

- An **agent harness** is this read-eval-print loop as one coherent piece of software¹ with:
 - **Tools:** read/edit/write files, run shell commands, search the codebase, (sometimes) the web
 - **Loop:** model proposes action → tool executes → output observed → repeat until done

¹Exactly what you built in the previous session!

Anatomy of a Coding Agent

- An **agent harness** is this read-eval-print loop as one coherent piece of software¹ with:
 - **Tools:** read/edit/write files, run shell commands, search the codebase, (sometimes) the web
 - **Loop:** model proposes action → tool executes → output observed → repeat until done
 - **Guardrails:** approval prompts, sandboxing, git

¹Exactly what you built in the previous session!

Anatomy of a Coding Agent

- An **agent harness** is this read-eval-print loop as one coherent piece of software¹ with:
 - **Tools:** read/edit/write files, run shell commands, search the codebase, (sometimes) the web
 - **Loop:** model proposes action → tool executes → output observed → repeat until done
 - **Guardrails:** approval prompts, sandboxing, git
- Example popular harnesses. All support external providers, so they can point at ALCF Inference:
 - **OpenCode** (what we'll demonstrate this session) and **Pi**, recommended in the ALCF docs
 - **Claude Code**, featured in the session right after this one
 - **Codex**, OpenAI's in-house harness for their GPT model series

¹Exactly what you built in the previous session!

Anatomy of a Coding Agent

- An **agent harness** is this read-eval-print loop as one coherent piece of software¹ with:
 - **Tools:** read/edit/write files, run shell commands, search the codebase, (sometimes) the web
 - **Loop:** model proposes action → tool executes → output observed → repeat until done
 - **Guardrails:** approval prompts, sandboxing, git
- Example popular harnesses. All support external providers, so they can point at ALCF Inference:
 - **OpenCode** (what we'll demonstrate this session) and **Pi**, recommended in the ALCF docs
 - **Claude Code**, featured in the session right after this one
 - **Codex**, OpenAI's in-house harness for their GPT model series
 - Additionally, there are community shims/proxies for anything else (see `alcf-proxy`, `llm-rosetta`)

¹Exactly what you built in the previous session!

The ALCF Inference Service



U.S. DEPARTMENT
of ENERGY

Argonne National Laboratory is a
U.S. Department of Energy laboratory
managed by UChicago Argonne, LLC.



Argonne Leadership
Computing Facility

ALCF Inference Endpoints at a Glance

- **30+ open-weight models** on ALCF hardware, behind one OpenAI-compatible API

²Streaming for these APIs is disabled on Sophia and Metis, use Chat Completions or Minerva instead

ALCF Inference Endpoints at a Glance

- **30+ open-weight models** on ALCF hardware, behind one OpenAI-compatible API
- The ALCF Inference Service is accessible with any DOE lab credentials via Globus; today you should authenticate using your **ALCF** login.

²Streaming for these APIs is disabled on Sophia and Metis, use Chat Completions or Minerva instead

ALCF Inference Endpoints at a Glance

- **30+ open-weight models** on ALCF hardware, behind one OpenAI-compatible API
- The ALCF Inference Service is accessible with any DOE lab credentials via Globus; today you should authenticate using your **ALCF** login.
- Web UI: <https://inference.alcf.anl.gov>, or query the API directly

²Streaming for these APIs is disabled on Sophia and Metis, use Chat Completions or Minerva instead

ALCF Inference Endpoints at a Glance

- **30+ open-weight models** on ALCF hardware, behind one OpenAI-compatible API
- The ALCF Inference Service is accessible with any DOE lab credentials via Globus; today you should authenticate using your **ALCF** login.
- Web UI: <https://inference.alcf.anl.gov>, or query the API directly
- Billions of LLM tokens for agents already served

²Streaming for these APIs is disabled on Sophia and Metis, use Chat Completions or Minerva instead

ALCF Inference Endpoints at a Glance

- **30+ open-weight models** on ALCF hardware, behind one OpenAI-compatible API
- The ALCF Inference Service is accessible with any DOE lab credentials via Globus; today you should authenticate using your **ALCF** login.
- Web UI: <https://inference.alcf.anl.gov>, or query the API directly
- Billions of LLM tokens for agents already served

Cluster	Hardware	Base path	Wire APIs
Sophia	NVIDIA A100	/resource_server/sophia/vllm/v1	chat, responses ² , messages ¹ , completions, embeddings
Metis	SambaNova SN40L	/resource_server/metis/api/v1	chat, responses ¹
Minerva	NVIDIA B200	/resource_server/minerva/api/v1	chat, responses, messages, completions, embeddings

²Streaming for these APIs is disabled on Sophia and Metis, use Chat Completions or Minerva instead

Picking a Model for Agents

- Agents talk to models through **tool calling**; the harness advertises tools (read file, run command, ...) and the model replies with structured calls

¹Artificial Analysis Intelligence Index v4.3.2 (current), each model at its best reasoning effort.

Picking a Model for Agents

- Agents talk to models through **tool calling**; the harness advertises tools (read file, run command, ...) and the model replies with structured calls
- Capability flags in the docs: **Batch**, **Reasoning**, **Tool calling**, always **Hot**

¹Artificial Analysis Intelligence Index v4.3.2 (current), each model at its best reasoning effort.

Picking a Model for Agents

- Agents talk to models through **tool calling**; the harness advertises tools (read file, run command, ...) and the model replies with structured calls
- Capability flags in the docs: **B**atch, **R**easoning, **T**ool calling, always **H**ot
- Agents need models with **T**; **R** improves planning; **H** avoids cold starts

¹Artificial Analysis Intelligence Index v4.3.2 (current), each model at its best reasoning effort.

Picking a Model for Agents

- Agents talk to models through **tool calling**; the harness advertises tools (read file, run command, ...) and the model replies with structured calls
- Capability flags in the docs: **B**atch, **R**easoning, **T**ool calling, always **H**ot
- Agents need models with **T**; **R** improves planning; **H** avoids cold starts

Model (subset)	Cluster	Flags	Score ¹	Notes
inkling-bf16	Minerva	RTH	25	975B MoE; recommended
nemotron-3-ultra	Minerva	RTH	23	550B MoE, 1M context; near-tie
openai/gpt-oss-120b	Sophia	BRT	12	batch + always hot; safe default
gpt-oss-120b	Minerva	RT	12	same model; not live on Minerva

¹Artificial Analysis Intelligence Index v4.3.2 (current), each model at its best reasoning effort.

Setting Up Your Harness



U.S. DEPARTMENT
of ENERGY

Argonne National Laboratory is a
U.S. Department of Energy laboratory
managed by UChicago Argonne, LLC.



Argonne Leadership
Computing Facility

Install the Agent

OpenCode

```
curl -fsSL https://opencode.ai/install | bash
```

Claude Code (next session!)

```
curl -fsSL https://claude.ai/install.sh | bash
```

reload your shell afterwards (zsh: ~/.zshrc, or open a new terminal)

```
source ~/.bashrc
```

²See the #agents section of the ALCF Inference docs for other agents

Install the Agent

```
# OpenCode
```

```
curl -fsSL https://opencode.ai/install | bash
```

```
# Claude Code (next session!)
```

```
curl -fsSL https://claude.ai/install.sh | bash
```

```
# reload your shell afterwards (zsh: ~/.zshrc, or open a new terminal)
```

```
source ~/.bashrc
```

- These one-liners install to ~/.local/bin.

²See the #agents section of the ALCF Inference docs for other agents

Install the Agent

OpenCode

```
curl -fsSL https://opencode.ai/install | bash
```

Claude Code (next session!)

```
curl -fsSL https://claude.ai/install.sh | bash
```

reload your shell afterwards (zsh: ~/.zshrc, or open a new terminal)

```
source ~/.bashrc
```

- These one-liners install to ~/.local/bin.
- Also available via brew, apt, etc.²

²See the #agents section of the ALCF Inference docs for other agents

Quick Configuration: alcf-ai

- alcf-ai writes generates a provider config that **refreshes your access token automatically**

```
# install uv if you don't have it  
curl -LsSf https://astral.sh/uv/install.sh | sh  
  
# authenticate once, then write the agent config  
uvx alcf-tokens login  
uvx alcf-ai agent configure opencode
```

Quick Configuration: alcf-ai

- alcf-ai writes generates a provider config that **refreshes your access token automatically**

```
# install uv if you don't have it  
curl -LsSf https://astral.sh/uv/install.sh | sh  
  
# authenticate once, then write the agent config  
uvx alcf-tokens login  
uvx alcf-ai agent configure opencode
```

- Supports opencode, pi, codex, and claude

Quick Configuration: alcf-ai

- alcf-ai writes generates a provider config that **refreshes your access token automatically**

```
# install uv if you don't have it
curl -LsSf https://astral.sh/uv/install.sh | sh

# authenticate once, then write the agent config
uvx alcf-tokens login
uvx alcf-ai agent configure opencode
```

- Supports opencode, pi, codex, and claude
- **Set once:** the agent fetches a fresh token on demand, you only need to re-run for new models or capabilities

Quick Configuration: `alcf-ai`

- `alcf-ai` writes generates a provider config that **refreshes your access token automatically**

```
# install uv if you don't have it
curl -LsSf https://astral.sh/uv/install.sh | sh

# authenticate once, then write the agent config
uvx alcf-tokens login
uvx alcf-ai agent configure opencode
```

- Supports opencode, `pi`, `codex`, and `claude`
- **Set once:** the agent fetches a fresh token on demand, you only need to re-run for new models or capabilities
- Need the raw token for a script?
`export ALCF_AI_TOKEN="$(uvx alcf-ai auth get-access-token)"`

What Just Got Configured?

```
{
  "provider": {
    "alcf-inference-service-sophia-vllm": {
      "npm": "@ai-sdk/openai-compatible",
      "options": {
        "baseUrl": "https://inference-api.alcf.anl.gov/resource_server/sophia/vllm/v1",
        "apiKey": "alcf-ai-auto-refresh"
      },
      "models": { "openai/gpt-oss-120b": {} }
    }
  },
  "plugin": [
    ["alcf-ai-opencode-plugin",
     { "tokenHelper": ["uvx", "alcf-tokens", "get-token", "inference"] }]
  ]
}
```

- OpenCode now knows about an ALCF provider (~/.config/opencode/opencode.jsonc).

What Just Got Configured?

```
{
  "provider": {
    "alcf-inference-service-sophia-vllm": {
      "npm": "@ai-sdk/openai-compatible",
      "options": {
        "baseUrl": "https://inference-api.alcf.anl.gov/resource_server/sophia/vllm/v1",
        "apiKey": "alcf-ai-auto-refresh"
      },
      "models": { "openai/gpt-oss-120b": {} }
    }
  },
  "plugin": [
    ["alcf-ai-opencode-plugin",
     { "tokenHelper": ["uvx", "alcf-tokens", "get-token", "inference"] } ]
  ]
}
```

- OpenCode now knows about an ALCF provider (~/.config/opencode/opencode.jsonc).
- The plugin supplies a fresh token per-request for OpenCode.

Working With the Agent



Argonne National Laboratory is a
U.S. Department of Energy laboratory
managed by UChicago Argonne, LLC.



Argonne Leadership
Computing Facility

Your First Session



The screenshot shows a terminal window with a dark background. At the top, the word "opencode" is displayed in a large, pixelated, light blue font. Below it, a dark rectangular box contains the text "Ask anything... 'Fix a TODO in the codebase'" in a light blue monospace font. Underneath this box, the text "Build ·inkling-bf16 ALCF Inference Service (Minerva, Direct · max API)" is shown in a light blue monospace font. Below the box, the text "tab agents ctrl+p commands" is displayed in a light blue monospace font. At the bottom left of the terminal, the prompt "~/ses-demo:main" is visible, and at the bottom right, the version number "1.18.31" is shown.

```
opencode
```

```
Ask anything... "Fix a TODO in the codebase"
```

```
Build ·inkling-bf16 ALCF Inference Service (Minerva, Direct · max API)
```

```
tab agents ctrl+p commands
```

```
~/ses-demo:main 1.18.31
```

Your First Session

A screenshot of a terminal window with a dark background. At the top, the word "opencode" is displayed in a large, white, pixelated font. Below it, a light gray rectangular box contains the text "Ask anything... 'Fix a TODO in the codebase'". Underneath this box, the command "Build ·inkling-bf16 ALCF Inference Service (Minerva, Direct · max API)" is shown in a monospaced font. Below the command box, the text "tab agents ctrl+p commands" is visible. At the bottom left of the terminal, the prompt "~ /ses-demo:main" is shown, and at the bottom right, the version number "1.18.31" is displayed.

```
opencode

Ask anything... "Fix a TODO in the codebase"

Build ·inkling-bf16 ALCF Inference Service (Minerva, Direct · max
                    API)

tab agents ctrl+p commands

~/ses-demo:main 1.18.31
```

- Run opencode inside your project!

Your First Session



```
opencode

Ask anything... "Fix a TODO in the codebase"

Build ·inkling-bf16 ALCF Inference Service (Minerva, Direct · max
                  API)

tab agents ctrl+p commands

~/ses-demo:main 1.18.31
```

- Run opencode inside your project!
- Ctrl+P lists every command

Talking to the Agent: @, !, and /

- @ adds a file or folder to the message by fuzzy search:

```
Refactor the error handling in @src/solve.py  
Compare our output to @reference/README.md
```

Talking to the Agent: @, !, and /

- @ adds a file or folder to the message by fuzzy search:

```
Refactor the error handling in @src/solve.py  
Compare our output to @reference/README.md
```

- ! runs a shell command and feeds the output back in:

```
!git log --oneline -5
```

Talking to the Agent: @, !, and /

- @ adds a file or folder to the message by fuzzy search:

```
Refactor the error handling in @src/solve.py  
Compare our output to @reference/README.md
```

- ! runs a shell command and feeds the output back in:

```
!git log --oneline -5
```

- / runs an agent command.

Talking to the Agent: @, !, and /

- @ adds a file or folder to the message by fuzzy search:

```
Refactor the error handling in @src/solve.py  
Compare our output to @reference/README.md
```

- ! runs a shell command and feeds the output back in:

```
!git log --oneline -5
```

- / runs an agent command.
 - /init writes an AGENTS.md that it reuses on later turns

Talking to the Agent: @, !, and /

- @ adds a file or folder to the message by fuzzy search:

```
Refactor the error handling in @src/solve.py  
Compare our output to @reference/README.md
```

- ! runs a shell command and feeds the output back in:

```
!git log --oneline -5
```

- / runs an agent command.
 - /init writes an AGENTS.md that it reuses on later turns
 - Try /models, /sessions, and /compact!

Watching the Agent Work

- Your prompt, the model's reasoning, and every **tool call** show as a transcript in your terminal
- Here Grep found the file and Read opened it, so you can audit exactly what it saw

```
Which file defines the solver, and what does it do? Look it up before you answer.
```

```
Thought: 4ms
```

```
The user asks: "Which file defines the solver, and what does it do? Look it up before you answer." We need to examine repo. Let's list files or search for "solver".
```

```
grep
```

```
* Grep "solver" (3 matches)
```

```
Thought: 1ms
```

```
Found files. Let's read diffusion.py.
```

```
→ Read diffusion.py
```

Plan First, Then Build

- Tab switches between the **Build** and **Plan** agents, the prompt box shows which is active
- **Plan** mode does its best to only analyze and propose changes

```
stability limit. Need to ask: what kind of warning? print? warnings module? Should it be
a warning or raise? Let's ask concise clarifying question.

Plan: Add a stability check inside diffusion.py:solve() before the loop, comparing dt to
stability_limit(dx, alpha).

Clarifying: Should it use print() or warnings.warn()? And should it raise an error or
just emit a warning?

■ Plan · inkling-bf16 · 2.3s

Plan · inkling-bf16 ALCF Inference Service (Minerva, Direct API) · max

/Users/ewong/ses-demo 8.6K (1%) ctrl+p commands
```

Subagents and Orchestration

- **Primary** agents are what you Tab between.
- **Subagents** are workers they delegate to, or that you call with `@explore`
- Built in: `general` (parallel units of work), `explore` (read-only code search), `scout` (upstream docs)

```
@explore where is the timestep computed, and which line updates the field?
```

```
Thought: 2ms
```

```
We need to answer user: "@explore where is the timestep computed, and which line updates the field?" This is ambiguous. Perhaps they refer to code in workspace. Let's explore workspace. Use explore agent to find timestep computation and field updates.
```

```
task
```

```
✓ Explore Task - Find timestep and field update  
  ↳ 16 toolcalls - 35.8s
```

```
ctrl+x down view subagents
```

```
Thought: 1ms
```

```
Only diffusion.py is relevant. Let's read it completely to report all line numbers for timestep computation and field updates. Also check if there are any hidden files with .py or other extensions that might have source.
```

```
$ find . -type f \( -name '*.py' -o -name '*.c' -o -name '*.cpp' -o -name '*.f' -o -name '*.for' -o -name '*.m' -o -name '*.js' -o -name '*.ts' -o -name '*.java' -o -name '*.scala' -o -name '*.go' -o -name '*.rs' \) -not -path './node_modules/*' -not -path './.opencode/*' -not -path './.git/*'  
  
./diffusion.py
```

```
Thought: 2ms
```

the delegate, and the worker's own session (Ctrl+X down)

Permissions

- Every tool is gated allow, ask, or deny — **Plan** is just this, with edits and bash set to ask

```
{  
  "permission": { "edit": "ask", "webfetch": "deny" },  
  "agent": { "reviewer": { "permission": {  
    "bash": { "*" : "ask", "git diff": "allow" } } } }  
}
```

Permissions

- Every tool is gated allow, ask, or deny — **Plan** is just this, with edits and bash set to ask

```
{  
  "permission": { "edit": "ask", "webfetch": "deny" },  
  "agent": { "reviewer": { "permission": {  
    "bash": { "*" : "ask", "git diff": "allow" } } } }  
}
```

- Bash rules take globs and the **last match wins**: put "*" : "ask" before "git diff": "allow"

Permissions

- Every tool is gated allow, ask, or deny — **Plan** is just this, with edits and bash set to ask

```
{  
  "permission": { "edit": "ask", "webfetch": "deny" },  
  "agent": { "reviewer": { "permission": {  
    "bash": { "*" : "ask", "git diff": "allow" } } } }  
}
```

- Bash rules take globs and the **last match wins**: put "*" : "ask" before "git diff": "allow"
- permission.task gates which subagents an agent may call; --auto approves everything you did not deny

Review the Diff

- Every edit displays as a **diff** with line numbers, just like in `git`
- Review the diff alongside summaries. Make incremental changes with each prompt, then test the code

```
← Edit diffusion.py

40  """Return (x, u, dt) after integrating `steps` timesteps."""
41  x = np.linspace(0.0, 1.0, n, endpoint=False)
42  u = initial_condition(x)
43  dt = 0.4 * stability_limit(dx, alpha)
44 + if dt > stability_limit(dx, alpha):
45 +     warnings.warn("dt exceeds stability limit")
46  for _ in range(steps):
47      u = step(u, alpha, dt, dx)
48  return x, u, dt
49
```

Undo and Restore

- `/undo` reverts the last change and hands your original prompt back so you can amend it³
- `/redo` (or `Ctrl+X r`) re-runs it

```
1 message reverted
ctrl+x r or /redo to restore

diffusion.py -4

Use warnings.warn() and only warn. Go ahead and make the change.

Build · inkling-bf16 ALCF Inference Service (Minerva, Direct API) · max

/Users/ewong/ses-demo 9.4K (1%) ctrl+p commands
```

³git stays your real undo button, though

Sessions and Sharing

- A **session** is one conversation

Sessions and Sharing

- A **session** is one conversation
 - /sessions switches between them

Sessions and Sharing

- A **session** is one conversation
 - /sessions switches between them
 - /new starts a clean one

Sessions and Sharing

- A **session** is one conversation
 - /sessions switches between them
 - /new starts a clean one
- /share publishes the transcript and copies a link, which you can import with `opencode import`

Sessions and Sharing

- A **session** is one conversation
 - /sessions switches between them
 - /new starts a clean one
- /share publishes the transcript and copies a link, which you can import with opencode import
 - Note: share links are **public**

Sessions and Sharing

- A **session** is one conversation
 - `/sessions` switches between them
 - `/new` starts a clean one
- `/share` publishes the transcript and copies a link, which you can import with `opencode import`
 - Note: share links are **public**
 - A project config can set `"share": "disabled"` to prevent this

Sessions and Sharing

- A **session** is one conversation
 - `/sessions` switches between them
 - `/new` starts a clean one
- `/share` publishes the transcript and copies a link, which you can import with `opencode import`
 - Note: share links are **public**
 - A project config can set `"share": "disabled"` to prevent this
- `--continue` and `--session resume` from the CLI

Sessions and Sharing

- A **session** is one conversation
 - `/sessions` switches between them
 - `/new` starts a clean one
- `/share` publishes the transcript and copies a link, which you can import with `opencode import`
 - Note: share links are **public**
 - A project config can set `"share": "disabled"` to prevent this
- `--continue` and `--session resume` from the CLI
- `--fork` branches so the original stays intact

Effective Vibecoding



Argonne National Laboratory is a
U.S. Department of Energy laboratory
managed by UChicago Argonne, LLC.



Argonne Leadership
Computing Facility

Iterate in Small Loops

- **Plan before you build:** ask the agent to read the code and propose a plan first; review it, then let it code

Iterate in Small Loops

- **Plan before you build:** ask the agent to read the code and propose a plan first; review it, then let it code
- **One feature per prompt:** small steps, run the code after every change, commit after every success

Iterate in Small Loops

- **Plan before you build:** ask the agent to read the code and propose a plan first; review it, then let it code
- **One feature per prompt:** small steps, run the code after every change, commit after every success
- **Don't fight a confused agent:** if it spirals, restate the goal with clearer context, or start a fresh session instead of piling on corrections

Iterate in Small Loops

- **Plan before you build:** ask the agent to read the code and propose a plan first; review it, then let it code
- **One feature per prompt:** small steps, run the code after every change, commit after every success
- **Don't fight a confused agent:** if it spirals, restate the goal with clearer context, or start a fresh session instead of piling on corrections
- Git is your undo button; a clean history of small commits makes every agent edit cheap to review or revert

Iterate in Small Loops

- **Plan before you build:** ask the agent to read the code and propose a plan first; review it, then let it code
- **One feature per prompt:** small steps, run the code after every change, commit after every success
- **Don't fight a confused agent:** if it spirals, restate the goal with clearer context, or start a fresh session instead of piling on corrections
- Git is your undo button; a clean history of small commits makes every agent edit cheap to review or revert

Manage the agent like guiding a very pedantic junior engineer! Small, well-defined tasks can be easily accomplished over broad, unscoped ideas.

Context is Everything

- Keep an **instructions file** (AGENTS.md, CLAUDE.md, ...) at the repo root: project purpose, build/run/test commands, conventions, and no-go zones ('never touch src/legacy')

Context is Everything

- Keep an **instructions file** (AGENTS.md, CLAUDE.md, ...) at the repo root: project purpose, build/run/test commands, conventions, and no-go zones ('never touch src/legacy')
- **Point, don't paste:** tell the agent which files matter and let it read/search the rest; pasted code goes stale

Context is Everything

- Keep an **instructions file** (AGENTS.md, CLAUDE.md, ...) at the repo root: project purpose, build/run/test commands, conventions, and no-go zones ('never touch src/legacy')
- **Point, don't paste:** tell the agent which files matter and let it read/search the rest; pasted code goes stale
- **Fresh session per task:** long transcripts rot the model's working memory (the context window) and the agent's judgment

Context is Everything

- Keep an **instructions file** (AGENTS.md, CLAUDE.md, ...) at the repo root: project purpose, build/run/test commands, conventions, and no-go zones ('never touch src/legacy')
- **Point, don't paste:** tell the agent which files matter and let it read/search the rest; pasted code goes stale
- **Fresh session per task:** long transcripts rot the model's working memory (the context window) and the agent's judgment
- Encode your **domain knowledge** explicitly: units, file formats, expected values, edge cases. The model cannot guess your physics.

Make Success Verifiable

- Agents shine when correctness is checkable; tests, linters, type checkers, a run script that prints a known result

Make Success Verifiable

- Agents shine when correctness is checkable; tests, linters, type checkers, a run script that prints a known result
- **Failing-test-first:** “write a test that captures the requirement, then make it pass”. Works extremely well for scientific codes

Make Success Verifiable

- Agents shine when correctness is checkable; tests, linters, type checkers, a run script that prints a known result
- **Failing-test-first:** “write a test that captures the requirement, then make it pass”. Works extremely well for scientific codes
- Paste error messages back verbatim; the agent can read tracebacks without needing to paraphrase

Make Success Verifiable

- Agents shine when correctness is checkable; tests, linters, type checkers, a run script that prints a known result
- **Failing-test-first:** “write a test that captures the requirement, then make it pass”. Works extremely well for scientific codes
- Paste error messages back verbatim; the agent can read tracebacks without needing to paraphrase
- For science: vetted outputs, sanity-limit checks, property tests. Verify figures against results you know.

Stay in Control

- **Review the work!** Ultimately, you are responsible for what ships (papers, repos, production)

Stay in Control

- **Review the work!** Ultimately, you are responsible for what ships (papers, repos, production)
- Be cautious and **mandate human intervention** for destructive or far-reaching tasks; sandbox the agent when you can

Stay in Control

- **Review the work!** Ultimately, you are responsible for what ships (papers, repos, production)
- Be cautious and **mandate human intervention** for destructive or far-reaching tasks; sandbox the agent when you can
- **Never paste tokens or credentials** into a prompt; keep secrets out of transcripts

Stay in Control

- **Review the work!** Ultimately, you are responsible for what ships (papers, repos, production)
- Be cautious and **mandate human intervention** for destructive or far-reaching tasks; sandbox the agent when you can
- **Never paste tokens or credentials** into a prompt; keep secrets out of transcripts
- Models **hallucinate**: invented APIs, imaginary flags, plausible-but-wrong physics. Verify imports, citations, and values. Don't blindly trust the agent!!!

Stay in Control

- **Review the work!** Ultimately, you are responsible for what ships (papers, repos, production)
- Be cautious and **mandate human intervention** for destructive or far-reaching tasks; sandbox the agent when you can
- **Never paste tokens or credentials** into a prompt; keep secrets out of transcripts
- Models **hallucinate**: invented APIs, imaginary flags, plausible-but-wrong physics. Verify imports, citations, and values. Don't blindly trust the agent!!!
- Know when not to vibe: unverifiable correctness-critical code, security-sensitive paths, etc.

Crash Course



U.S. DEPARTMENT
of ENERGY

Argonne National Laboratory is a
U.S. Department of Energy laboratory
managed by UChicago Argonne, LLC.

Argonne
NATIONAL LABORATORY



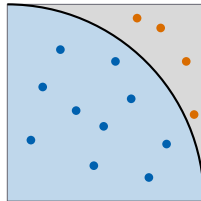
Argonne Leadership
Computing Facility

Exercise: Vibecoded Monte Carlo π

- Throw N random darts at the unit square; the fraction p landing in the quarter circle estimates its area:

$$\hat{\pi} = 4p, \quad \sigma_{\hat{\pi}} = 4\sqrt{p(1-p)/N}$$

- Small enough to finish in one session, yet it has everything real science code has: numerics, an error bar, a known answer, and a figure
- Same loop every step: **prompt** $\rightarrow$ **code** $\rightarrow$ **python pi_mc.py** $\rightarrow$ **look at the plot** $\rightarrow$ **refine**
- You are the architect; the agent is the executor; π is the referee



$$\hat{\pi} = 4 \times \text{inside} / \text{all}$$

Environment Setup

- **Checklist:** setup.sh has already installed uv, OpenCode, and the Python environment, logged you in, and set up OpenCode for ALCF

```
cd Service_Enabled_Science
./setup.sh                # safe to re-run; sets up env + agents
source .venv/bin/activate # numpy + matplotlib now on PATH

mkdir -p ~/pi-demo && cd ~/pi-demo
git init                  # your undo button
opencode                  # pick inkling-bf16 (Minerva)
```

Environment Setup

- **Checklist:** `setup.sh` has already installed `uv`, `OpenCode`, and the Python environment, logged you in, and set up `OpenCode` for ALCF

```
cd Service_Enabled_Science
./setup.sh                # safe to re-run; sets up env + agents
source .venv/bin/activate  # numpy + matplotlib now on PATH

mkdir -p ~/pi-demo && cd ~/pi-demo
git init                  # your undo button
opencode                  # pick inkling-bf16 (Minerva)
```

- Use a new, empty directory so the agent can't find the reference `solution.py`

Environment Setup

- **Checklist:** `setup.sh` has already installed `uv`, `OpenCode`, and the Python environment, logged you in, and set up `OpenCode` for ALCF

```
cd Service_Enabled_Science
./setup.sh                # safe to re-run; sets up env + agents
source .venv/bin/activate # numpy + matplotlib now on PATH

mkdir -p ~/pi-demo && cd ~/pi-demo
git init                  # your undo button
opencode                  # pick inkling-bf16 (Minerva)
```

- Use a new, empty directory so the agent can't find the reference `solution.py`
- The full prompt and the follow-ups are in `04_Coding_with_Inference_Service/PROMPT.md`

Anatomy of a Good Prompt

Write a single Python script, `pi_mc.py`, that estimates π with Monte Carlo sampling and makes a publication-quality figure of the result.

Environment:

- Use the `python` already on PATH (numpy + matplotlib). Do not pip install anything.
- Headless: savefig only, never `plt.show()`.

The math:

- `N` uniform points, `np.random.default_rng(seed), vectorized; pi_hat = 4p; stderr = 4*sqrt(p(1-p)/N).`
- Print the estimate $\pm$ stderr; fail if it is more than 3 stderr from `math.pi`.

The figure: (1) dartboard scatter, (2) running estimate with a ± 2 sigma band on a log x-axis, (3) log-log error vs. a $\sigma/\sqrt{N}$ line.

Then run it twice, show me the output, and fix any errors before you report back.

- **Name the deliverable:** one file, one output

Anatomy of a Good Prompt

Write a single Python script, `pi_mc.py`, that estimates π with Monte Carlo sampling and makes a publication-quality figure of the result.

Environment:

- Use the `python` already on PATH (numpy + matplotlib). Do not pip install anything.
- Headless: savefig only, never `plt.show()`.

The math:

- N uniform points, `np.random.default_rng(seed), vectorized;  $\pi_{\text{hat}} = 4p$ ;  $\text{stderr} = 4\sqrt{p(1-p)/N}$ .`
- Print the estimate $\pm$ stderr; fail if it is more than 3 stderr from `math.pi`.

The figure: (1) dartboard scatter, (2) running estimate with a ± 2 sigma band on a log x-axis, (3) log-log error vs. a $\sigma/\sqrt{N}$ line.

Then run it twice, show me the output, and fix any errors before you report back.

- **Name the deliverable:** one file, one output
- **Pin the environment:** open models guess the environment wrong unless you tell them what's installed

Anatomy of a Good Prompt

Write a single Python script, `pi_mc.py`, that estimates π with Monte Carlo sampling and makes a publication-quality figure of the result.

Environment:

- Use the `python` already on PATH (numpy + matplotlib). Do not pip install anything.
- Headless: savefig only, never `plt.show()`.

The math:

- N uniform points, `np.random.default_rng(seed), vectorized;  $\pi_{\text{hat}} = 4p$ ;`
`stderr = 4*sqrt(p(1-p)/N).`
- Print the estimate $\pm$ stderr; fail if it is more than 3 stderr from `math.pi`.

The figure: (1) dartboard scatter, (2) running estimate with a ± 2 sigma band on a log x-axis, (3) log-log error vs. a $\sigma/\sqrt{N}$ line.

Then run it twice, show me the output, and fix any errors before you report back.

- **Name the deliverable:** one file, one output
- **Pin the environment:** open models guess the environment wrong unless you tell them what's installed
- **Spell out the science:** formulas, units, and expected values; the model struggles with niche domain knowledge

Anatomy of a Good Prompt

Write a single Python script, `pi_mc.py`, that estimates π with Monte Carlo sampling and makes a publication-quality figure of the result.

Environment:

- Use the `python` already on PATH (numpy + matplotlib). Do not pip install anything.
- Headless: savefig only, never `plt.show()`.

The math:

- N uniform points, `np.random.default_rng(seed)`, vectorized; `pi_hat = 4p`; `stderr = 4*sqrt(p(1-p)/N)`.
- Print the estimate $\pm$ stderr; fail if it is more than 3 stderr from `math.pi`.

The figure: (1) dartboard scatter, (2) running estimate with a ± 2 sigma band on a log x-axis, (3) log-log error vs. a $\sigma/\sqrt{N}$ line.

Then run it twice, show me the output, and fix any errors before you report back.

- **Name the deliverable:** one file, one output
- **Pin the environment:** open models guess the environment wrong unless you tell them what's installed
- **Spell out the science:** formulas, units, and expected values; the model struggles with niche domain knowledge
- **Describe the figure panel by panel:** “make it pretty” gives a generic plot

Anatomy of a Good Prompt

Write a single Python script, `pi_mc.py`, that estimates π with Monte Carlo sampling and makes a publication-quality figure of the result.

Environment:

- Use the `python` already on PATH (numpy + matplotlib). Do not pip install anything.
- Headless: savefig only, never `plt.show()`.

The math:

- N uniform points, `np.random.default_rng(seed), vectorized;  $\pi_{\text{hat}} = 4p$ ;  $\text{stderr} = 4\sqrt{p(1-p)/N}$ .`
- Print the estimate $\pm \text{stderr}$; fail if it is more than 3 stderr from `math.pi`.

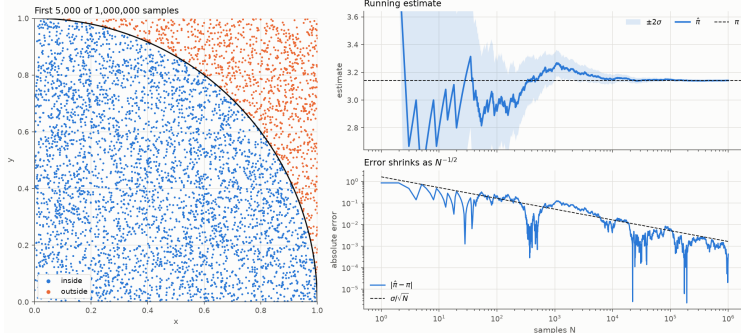
The figure: (1) dartboard scatter, (2) running estimate with a ± 2 sigma band on a log x-axis, (3) log-log error vs. a $\sigma/\sqrt{N}$ line.

Then run it twice, show me the output, and fix any errors before you report back.

- **Name the deliverable:** one file, one output
- **Pin the environment:** open models guess the environment wrong unless you tell them what's installed
- **Spell out the science:** formulas, units, and expected values; the model struggles with niche domain knowledge
- **Describe the figure panel by panel:** “make it pretty” gives a generic plot
- **Build in a check:** the 3σ assert and “run it” let the agent fix its own mistakes

What Good Looks Like

Monte Carlo $\pi = 3.14117 \pm 0.00164$ (true $\pi = 3.14159$)



10^6 samples: $\hat{\pi} = 3.14117 \pm 0.00164$, which is 0.26σ from π . The error follows the $N^{-1/2}$ line, so each additional digit of π takes $100\times$ more samples.

Check the Agent's Work, Then Push Further

Verify

- Run it yourself: 10^6 samples should give an estimate within ~ 0.003 of π
- Open the PNG: dots split at the arc, the estimate settles onto π , the error slope matches $N^{-1/2}$
- Read the diff: no Python loop over samples, no `plt.show()`, no `pip install`
- Commit once it works

Check the Agent's Work, Then Push Further

Verify

- Run it yourself: 10^6 samples should give an estimate within ~ 0.003 of π
- Open the PNG: dots split at the arc, the estimate settles onto π , the error slope matches $N^{-1/2}$
- Read the diff: no Python loop over samples, no `plt.show()`, no `pip install`
- Commit once it works

Then, one prompt at a time

- Run K independent seeds: is their spread consistent with the standard error?
- Time $N = 10^5 \dots 10^8$ and report how runtime scales
- Add a `pytest` check against π for several seeds
- Swap in a Sobol sequence (`scipy.stats.qmc`): how much faster does the error fall?

Going Further



U.S. DEPARTMENT
of ENERGY

Argonne National Laboratory is a
U.S. Department of Energy laboratory
managed by UChicago Argonne, LLC.



Argonne Leadership
Computing Facility

Skills, MCP, Plugins, and more

- A **skill** is a folder with a `SKILL.md`: the agent sees only its name, description, and can call the `skill` tool when a task matches
- An **MCP server** adds server-provided tools, declared in `opencode.jsonc` under `mcp`
- **Plugins** are JS/TS that extends `opencode`'s own events, `alcf-ai-opencode-plugin` is one example
- Misc built-ins: `"lsp": true` gives the agent diagnostics, `"formatter": true` formats each edit

```
Load the plot-style skill, then check diffusion.py against it.
```

```
Thought: 1ms
```

```
The user wants me to load the plot-style skill and then check diffusion.py against it.  
Let me start by loading the skill.
```

```
skill
```

```
+ Skill "plot-style"
```

```
Thought: 1ms
```

```
Now I need to find and read diffusion.py.
```

Skills, MCP, Plugins, and more

- A **skill** is a folder with a `SKILL.md`: the agent sees only its name, description, and can call the `skill` tool when a task matches
- An **MCP server** adds server-provided tools, declared in `opencode.jsonc` under `mcp`
- **Plugins** are JS/TS that extends `opencode`'s own events, `alcf-ai-opencode-plugin` is one example
- Misc built-ins: `"lsp": true` gives the agent diagnostics, `"formatter": true` formats each edit

Next session uses Claude Code to demonstrate these capabilities! MCP in `.mcp.json`, skills in `~/.claude/skills/`.

```
Load the plot-style skill, then check diffusion.py against it.
```

```
Thought: 1ms
```

```
The user wants me to load the plot-style skill and then check diffusion.py against it.  
Let me start by loading the skill.
```

```
skill
```

```
+ Skill "plot-style"
```

```
Thought: 1ms
```

```
Now I need to find and read diffusion.py.
```

References

- ALCF Inference Endpoints docs (incl. #agents):
<https://docs.alcf.anl.gov/services/inference-endpoints>
- ALCF Inference Endpoints repo & examples:
<https://github.com/argonne-lcf/inference-endpoints>
- alcf-ai on PyPI: <https://pypi.org/project/alcf-ai/>
- Harness homepages: OpenCode <https://opencode.ai/>; Pi <https://pi.dev/>; Claude Code <https://code.claude.com/docs/en/quickstart>; Codex <https://learn.chatgpt.com/docs/codex/cli>
- Karpathy, A. (Feb 2025): “vibe coding” post on X
- Tanikanti et al., *FIRST: Federated Inference Resource Scheduling Toolkit for Scientific AI Model Access*, SC Workshops '25. <https://doi.org/10.1145/3731599.3767346>
- Mailing list: inference-service-notify (<https://lists.alcf.anl.gov/mailman/listinfo/inference-service-notify>)